Accuracy and Efficiency Tradeoffs of Random Forest, Support Vector Machine, and k-Nearest Neighbour Classifiers on Two Datasets, and the Effects of Sample Reduction

Antonio Gutiérrez, Arnau Martínez, Lucas King, Miguel Ruiz
Course: Introduction to Machine Learning
Master in Artificial Intelligence

October 25, 2026

1 Introduction

In this investigation, accuracy, time efficiency, and storage metrics are used to evaluate classification models corresponding to two paradigms of supervised machine learning: lazy, instance-based methods that defer computation until inference time; and eager, model-based algorithms that learn decision boundaries at training time. This work presents a hyperparameter analysis and comparative study of three classifiers, evaluated across the Wine Quality [1] and Pen-based [2] UCI datasets using ten pre-partitioned cross-validation folds per dataset. The models investigated are the lazy learning k-Nearest Neighbors (k-NN) classifier and two eager learning classifiers: the Support Vector Machine Classifier (SVC), and Random Forest (RF) Classifier.

The examination of k-NN tests combinations of three different k values (3,5, and 7), four distance metrics (Euclidean, Manhattan, Canberra, and IVDM [3]), two voting schemes (majority and inverse-distance), and four dynamic test-time case-base maintenance policies: no retention (Never), unconditional retention (Always), error-driven Different Class retention (DC), and adaptive forgetting via Oblivion by Goodness (OG). The SVC evaluation examines both linear and radial basis function (RBF) kernels, in combination with different values of C, the misclassification penalty during fitting, and gamma, which determines the radius of influence that an instance exerts in an RBF kernel. The RF investigation examines both gini and entropy node splitting criteria, forest sizes of 150 and 300, as well as three choices for maximum tree depth, and binary choices for the minimum number of samples in a leaf node and the maximum features considered at each node of a tree. We refer to these tests before sample reduction as baseline tests. Beyond these, we investigate the impact of training-set instance reduction across the top-performing classifier of each classifier type by implementing three sample reduction algorithms, employing condensation, edition, and hybrid approaches respectively: Fast Condensed Nearest Neighbor (FCNN1) [4], Edited Nearest Neighbor with Probability Threshold (ENNTh) [5], and Decremental Reduction Optimization Procedure 4 (DROP4) [6].

We follow a two-stage evaluation pipeline. First, we select the optimal baseline configuration per classifier family using the highest mean accuracy, breaking ties by highest time efficiency. Second, we pair the three selected models with all four reduction options (no reduction as well as FCNN1, ENNTh, and DROP4), yielding 12 final candidates per dataset. We then compare all candidate pipelines (evaluating classification accuracy, per-instance inference latency, training-set storage retention, and preparation time) and finally assess statistical significance across the ten folds via a Friedman omnibus test followed by Holm-corrected pairwise post-hoc rank comparisons, highlighting the trade-offs between predictive performance, computational speed, and memory footprint.

2 Materials and Data Processing

To evaluate the classification models, two benchmark datasets from the UCI Machine Learning Repository were employed: Wine Quality [1] and Pen-Based Recognition of Handwritten Digits [2]. The Wine dataset consists of chemical analysis measurements of wines produced from three different cultivars grown in the same region of Italy. It contains 178 instances characterized by 13 continuous, real-valued attributes representing physicochemical properties. The target variable consists of three labels, comprising 59, 71, and 48 samples – a roughly balanced class distribution overall. On the other hand, the Pen-Based dataset comprises 10992 instances, capturing pen-based handwritten digit, with 10 balanced digit classes, encoded as integers 0, 1, ..., 9, each representing approximately 10% of the total dataset (roughly 1100 instances per class).

Both benchmarks are evaluated using a pre-partitioned 10-fold stratified cross-validation protocol to maintain representative class distributions. For the Wine dataset, each fold allocates approximately 160 training instances and 18 test instances (ranging from 16 to 19). For the Pen-Based dataset, training folds contain approximately 9893 instances, while test folds contain on average 1099 instances (ranging from 1,097 to 1,102).

Prior to model training and evaluation, min-max normalization was applied to map all continuous features into the range [0, 1], ensuring equal weighting across attributes for distance-sensitive learners (such as k-NN and SVM). Moreover, in order to guarantee methodological validity and strictly prevent data leakage, the transformation parameters, which include the feature-wise minimum and peak-to-peak range, were estimated solely from the training fold and subsequently used to normalize both training and test sets.

One consideration regarding normalised test features was the possibility of their values falling outside the [0, 1] range in the case of lower minima or higher maxima than had appeared in training folds. We opted not to clip these values to the [0, 1] range in order to not lose information regarding the magnitude of these outliers.

Neither dataset contained missing feature values, although an imputation protocol was implemented: for a given numerical feature, missing values in the train and test set would each be replaced by the feature's mean value in the train set; for a given nominal feature, missing values would be replaced by the feature's modal value in the train set.

Neither dataset contained nominal features. Had this been the case, each feature would have been encoded using integer values corresponding to distinct classes, with the intention to keep dimensionality of the feature space low and thus avoid the "curse of dimensionality" that can degrade the performance of k-NN models in particular.

3 Support Vector Machine Classifiers

The first of the classification algorithms used in this project is the Support Vector Machine Classifier. This is a supervised learning algorithm that learns decision boundaries to separate classes. During training, the Support Vector Machine seeks a decision boundary with a wide margin between classes. The training instances that determine this boundary are called support vectors. Once trained, the model uses the learned boundaries to predict the class of new instances.

To implement the Support Vector Machine, we used the SVC class from the Python scikit-learn library [7]. We tested two kernels: linear and Radial Basis Function (RBF). A kernel is a function that compares instances and influences how the model separates the classes. The linear kernel uses hyperplane decision boundaries, while RBF allows curved boundaries and is more flexible. For the linear kernel, we tested different values of C, while for the RBF kernel, we varied both C and gamma. C controls how much the model penalizes training instances that fall inside the margin or are classified incorrectly. Gamma is used with the RBF kernel and controls how far the influence of each training instance reaches.

The different hyperparameter values evaluated for the SVC are presented in Table 1. For the linear kernel, we tested five values of C. For the RBF kernel, we tested six values of C combined with five gamma settings, resulting in a total of 35 configurations per dataset.

Table 1: Hyperparameter values evaluated for SVC on both datasets.

Kernel	**C values**	**Gamma values**	**Configs.**
Linear	0.01, 0.1, 1, 10, 100	Not applicable	5
RBF	0.01, 0.1, 1, 10, 100, 1000	scale, 0.001, 0.01, 0.1, 1	30
Total			**35**

We evaluated each configuration using the 10 predefined folds for each dataset. For each fold, we trained the model and recorded its accuracy and prediction time per instance on the test set. We then calculated the average results across the 10 folds to compare the different configurations.

After comparing the results, we selected the configuration with the highest average accuracy for each dataset. If two or more configurations had the same average accuracy, we chose the one with the lowest average prediction time per instance. This gave us one selected SVC configuration for Wine and another for Pen-based.

4 Random Forest Classifiers

The second supervised learning classifier used in this project is the Random Forest (RF) classifier [8]. Like SVC, RF is an example of an eager learner, as it constructs a global prediction function at training time. Random Forest uses bootstrap aggregating, or bagging, to train multiple decision trees on random samples of the training data. These samples are drawn with replacement. Randomly selecting features at each split helps reduce the correlation between the trees. At prediction time, the classifier combines the class probabilities estimated by the trees and selects the class with the highest average probability.

For this project, we use the RandomForestClassifier class of the Python scikit-learn library. We test different values for five hyperparameters: n_estimators determines how many trees the Random Forest will build; criterion decides the quality of the tree split; max_depth controls the maximum depth of the trees; min_samples_leaf determines the minimum number of instances that each leaf must contain; and max_features determines how many features to consider at each node split within a tree.

The different hyperparameter values evaluated for the Random Forest are presented in Table 2. We tested different combinations of the five hyperparameters, resulting in a total of 48 configurations per dataset.

Table 2: Hyperparameter values evaluated for RF on both datasets.

Hyperparameter	**Tested values**
n_estimators	150, 300
criterion	gini, entropy
max_depth	None, 20, 40
min_samples_leaf	1, 2
max_features	sqrt, 0.7
Total configurations	**48**

We evaluated each configuration using the 10 predefined folds for each dataset. For each fold, we trained the model and recorded its accuracy and prediction time per instance on the test set. We then calculated the average results across the 10 folds to compare the different configurations.

After comparing the results, we selected the configuration with the highest average accuracy for each dataset. If two or more configurations had the same average accuracy, we chose the one with the lowest average prediction time per instance. This gave us one selected RF configuration for Wine and another for Pen-based.

5 k-Nearest Neighbour Classifiers

The final classification algorithm assessed is the k-Nearest Neighbours (kNN) algorithm. This is a supervised lazy learning algorithm that predicts the class of a query instance using the labels of its k nearest neighbours in the training set, according to some distance metric relative to the feature space.

Our implementation of a kNN classifier is tested with various configurable parameters: the value of k; the distance metric to measure similarity between instances; the voting policy to arrive at a classification for a query; and the retention policy to determine which instances should remain in the model's stored set.

5.1 Values of k

We tested three values of k: 3, 5, and 7. With a smaller k, the model uses fewer neighbours to predict the class; with a larger value, it considers more instances, with proportionally more computation, and therefore time, required at consultation. The value of k also has implications regarding the accuracy of the model. Low values consider only the immediate neighbours of a point and can lead to overfitting, characterised by low model bias and high model variance. Conversely, higher values cause a model to consult more points to reach a classification, which can lead to underfitting with high bias and low variance.

5.2 Distance Metrics

We also tried four distance metrics: Euclidean, Manhattan, Canberra and the Interpolated Value Distance Metric (IVDM). Each distance evaluates the separation between two points $\mathbf{x} = \{x_1, ..., x_n\}$ and $\mathbf{y} = \{y_1, ..., y_n\}$ in an n-dimensional feature space.

The Euclidean distance, or Minkowski with order 2, measures the straight line distance between two instances according to the following formula:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

The Manhattan distance sums the absolute difference for each feature according to:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} |x_i - y_i|$$

The Canberra distance considers how large differences in feature values are compared with the values themselves:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \frac{|x_i - y_i|}{|x_i + y_i|}$$

IVDM was selected for trial over Heterogenous Euclidean-Overlap Metric (HEOM) [3] as HEOM can be expected to behave exactly as Euclidean distance given the normalised value ranges for each feature and the lack of nominal values in both datasets.

IVDM considers, per feature of an instance, the likelihood of the instance belonging to a particular class given that feature's value. $p_{a,c}(x_a)$ is the calculated conditional probability of an instance $\mathbf{x}$ with value x_a for some feature, a, having class c. The distance between x_a and the midpoint of the discrete bin it falls into in the range of the feature's values, alongside per-class probabilities for the bin, are used to interpolate this figure — the finer details are available in Wilson and Martínez' paper [3]. At a higher level, the IVDM distance for feature a between instances $\mathbf{x}$ and $\mathbf{y}$ with feature values x_a and y_a is given by:

$$ivdm_a(x_a, y_a) = \sum_{c=1}^{C} |p_{a,c}(x_a) - p_{a,c}(y_a)|^2$$

where C is the set of all classes in the training set. The overall distance between $\mathbf{x}$ and $\mathbf{y}$ is then given by:

$$IVDM(\mathbf{x}, \mathbf{y}) = \sum_{a=1}^{n} ivdm_a(x_a, y_a)^2$$

where n is the total number of features.

5.3 Voting Policies

Two voting policies were implemented: majority label and inverse distance. The majority label policy reaches a consensus from the k nearest neighbours of a query by selecting the most-represented label among them to return as its final classification. The inverse distance policy uses a weighted sum of occurrences of each class among the k neighbours according to the inverse of each neighbour's distance from the query. This distance is calculated according to the specified metric. In the case of a tie between two or more candidates in either policy, the other policy was used as a tie break. If another tie occurred during the tie-break, the earliest entry, per the original training set order, was selected.

5.4 Retention Policies

Besides the implementation of the standard kNN algorithm, some instance-retention policies, commonly employed in lazy learning algorithms such as Case-based reasoning (CBR), have been implemented to dynamically adjust the size of the stored instance set based on the performance of the instances processed.

The simplest approach is Always retain, which appends every newly evaluated instance to the training set to inform subsequent predictions. While expanding the number of instances enriches the description of the search space, it simultaneously increases the query time and might introduce redundancy.

On the other hand, Different class retention (DC) selectively incorporates misclassified cases, focussing on only storing new instances that expose the current reference set's insufficiency. The approach is used during the training phase in the IB2 algorithm [9], with the rationale that misclassified instances overwhelmingly reside near concept boundaries. Storing only these instances and ignoring redundant core instances adds information about a class' boundaries while saving memory and computation.

Lastly, one "forgetting" policy, Oblivion by Goodness (OG), was implemented, to prune uninformative instances rather than add new ones. By tracking the predictive utility of each instance via a goodness parameter, which measures whether the contributions of the k nearest neighbors align with the final classification label, the algorithm is able to remove that instance when its goodness score falls below a threshold.

6 Combining sample reduction techniques with highest-performing classifiers

In addition to accuracy and efficiency, another key metric – with implications regarding the previous two – is storage. Lazy learning models have storage requirements that are proportional to the size of their training sets, which means larger sets can become highly demanding on resources. Sample reduction techniques aim to reduce storage requirements, reduce computation at consultation time, increase tolerance to noise, and maintain accuracy while selecting only a subset of the original training set for storage, rather than its entirety [6]. In this phase, we consider one algorithm from each of the condensation, edition, and hybrid feature reduction paradigms: FCNN [4], ENNTh [5], and DROP4 [6] respectively. We assess the impact of each reduction algorithm through analysing changes in accuracy, efficiency, and storage metrics for the highest-performing RF, SVM, and k-NN classifiers on each dataset, first using each algorithm to reduce all folds.

6.1 FCNN

FCNN approach is an extension of the Condensed Nearest Neighbour (CNN) rule originally proposed by Hart [10]. It incrementally builds a training-set consistent subset, denoted as S, by selectively retaining only the instances necessary to define precise decision boundaries between classes. The FCNN1 implementation initializes this search space by selecting centroids of each class to serve as the starting prototypes for S.

During each iteration, the algorithm partitions the feature space into Voronoi cells, which represents a region where all contained points or instances are physically closer to its governing prototype in S than to any other. Within each cell, the algorithm searches for Vornoi enemies, which Hart et. al. defines as data points whose true class label differs from that of the cell's prototype. To resolve these classification conflicts, FCNN1 selects the Voronoi enemy nearest to the prototype and promotes it to become a new prototype in S.

This iterative process dynamically redraws the feature space, which places new prototypes directly along the class boundaries. The end criterion for the algorithm to consider the process to be converged, is achieved whenever no Voronoi enemies remain. Hence, every cell is completely label-pure and the entire training set is correctly classified. Therefore, the process aims to minimize the number of retained instances and avoids an over-description of the interior feature space.

6.2 ENNTh

This method is an adaptation of Wilson's Edited Nearest Neighbour algorithm [11], which filters out noise from boundaries between classes in the training set, preserving points in the interior of a class' region in the feature space. It is a batched edition algorithm: it begins with the full training set, selects points for filtration according to some criteria, and removes them all together at the end of the run. In their filtration, Vásquez et al. [5] classify points in the training set by selecting the class with the highest probability of corresponding to the point. If the point is misclassified, it is discarded. Additionally, we have adopted their thresholding technique of setting some $\mu, 0 < \mu < 1$, which is used to discard points that don't correspond to any class with a probability above the threshold. Based on their report, an aggressive μ value of 0.8 outperformed the other tested values of 0.6 and 0.7 in terms of sample reduction size and accuracy on almost all of the 14 UCI datasets used. Other parameters not specified by the authors were k, the number of nearest neighbours upon which to base the class probability distribution, and the nature of the distance measure. In our implementation, we opted for Euclidean distance and k-values corresponding to the most performant k-NN model on each dataset.

6.3 DROP4

Decremental Reduction Optimization Procedure 4 is a hybrid sample reduction algorithm developed by Wilson and Martínez [6]. It serves as an extension of their algorithms DROP1-3. The reduction occurs in two passes:

Firstly, it iterates through training instances and establishes whether they are misclassified by their neighbours. As an added precaution, it then checks whether removing the instance would cause more of its associates (other instances which have the current instance as a neighbour) to be misclassified than would be currently. If the instance has been misclassified and the number of its associates' correct classifications would not decrease without it, it is removed.

The remaining instances are then sorted in decreasing order by distance to their nearest enemy, that is, the nearest instance with a different class label. The second and final reduction pass iterates over the sorted instances and solely considers whether their removal would cause a decrease in the number of correct classifications of its associates in the *original* training set. If no decrease is observed, the instance is removed.

7 Results

7.1 Protocol and comparison phases

This study compares kNN, SVC, and Random Forest (RF) in terms of accuracy, prediction time, and storage requirements. It also examines how their performance changes when the number of training instances is reduced. We use the Wine and Pen-based datasets, which have 178 and 10,992 instances respectively, along with the ten training and test splits provided for the project. Each dataset is analyzed separately which means that the results may vary from one dataset to the other.

We divided the evaluation into two stages. First, we compared different configurations within each model family. Then, we compared the selected models, both with and without instance reduction. Keeping these stages separate was useful to avoid mixing hyperparameter selection with the final comparison.

Table 3: The two phases of analysis

Stage	**Candidates compared**	**Output**
1. Select configuration	RF: 48; SVC: 35; kNN: 96	One configuration for each model family, for each dataset
2. Models and reductions	RF, SVC and kNN (best config only) × no reduction, FCNN, ENNTh and DROP4	12 candidates per dataset producing 120 measurements per dataset.

In the first stage, every configuration is evaluated across all ten given folds. This gives 1,790 measurements per dataset ($48 + 35 + 96$ configurations $\times$ 10 folds). In the second stage, we keep the selected hyperparameters fixed. We evaluate the models without reduction again, alongside their reduced versions. The final comparisons use the prediction times recorded in the second stage, not those measured during hyperparameter selection.

During preprocessing, medians, minimum values, and ranges are calculated using only the training data. The resulting transformations are then applied to the test data, keeping model fitting and evaluation separate [12]. We do not apply clipping in these experiments. This means test values that fall outside the training range are preserved, even if some normalized values end up outside $[0, 1]$. This is a design choice, and its implications are discussed in the limitations. Accuracy and prediction time are analyzed separately. We do not combine them into a single score.

It is important to note that RF and SVC predict the entire test set in a batch, but kNN processes test cases in their original order and updates its case base after making a prediction and learning the true label of the current case. Its measured time includes retaining or forgetting instances. We limited using the harness for each model to work on a single thread and all tests were run on the same computer, a 2020 MacBook Pro with an M1 Apple Silicon chip, under the same conditions.

7.2 How do we assess evidence of differences?

Our analysis was mainly based on Demšar [13]. We used a Friedman test to compare multiple candidates and a post hoc test to identify which pairs differ. We used this approach in both phases of the analysis, selecting a final candidate for each dataset using the accuracy-first rule described below.

The Friedman test works with ranks. Within each fold, the best candidate receives rank 1, the next best rank 2, and so on. Tied candidates receive the average of their rank positions. Each candidate then receives an average of its ranks across all folds. The Friedman test is an omnibus test, with the null hypothesis that all candidate configurations perform equally, with differences solely due to chance. A significant result indicates that at least one candidate differs in perfomance from the others.

We use a significance level of 0.05. A p-value below this threshold provides evidence of differences, provided the assumptions of the procedure hold. This does not tell us the magnitude of performance difference, and a non-significant result does not prove that the candidates perform equally well.

It was decided that each fold should occupy the position that a dataset would have in Demšar's comparison. However, Demšar's paper uses independent

datasets, whereas our folds share training observations (see Demšar [13], p. 5 and Section 3.2.3). We discuss the implications of this decision later.

We estimate the Friedman p-value by randomly shuffling candidate labels within each fold 9,999 times. For each of these shuffled permutations, ranks are computed, and the Friedman statistic is calculated. The p-value is calculated as $(b + 1)/(9,999 + 1)$, where b is the number of permutations with a rank separation at least as large as the observed one. The smallest attainable p-value is 0.0001. The random seed was set at 42 to make the results reproducible.

The paper presents the chi-squared approximation and the Iman–Davenport correction (Demšar [13], Section 3.2.2). Our code stores the chi-squared p-value for reference, but decisions are based on the permutation p-value.

If the Friedman test is significant for a given metric, we compare the average ranks of every pair of candidates using two-sided tests based on a normal distribution approximation, with a standard error that accounts for ties. Each pair's difference in average rank receives a z-statistic and p-value relative to this standard error. At this stage, we consider family-wise error: the increased likelihood of false positive results when a family of tests is large. To account for this, we apply Holm's step-down correction to the resulting p-values from the tests, ordered from most to least significant. We select a family-wise significance level of $\alpha = 0.05$. In general, for p-values ordered smallest to largest:

$$p_1 \leq p_2 \leq ... \leq p_m$$

The post-hoc test iterates over each p-value, which must satisfy:

$$p_i < \frac{\alpha}{m - i + 1}$$

Where m is the total number of pairs. If the inequality holds, the difference in average ranks in the pair is sufficient to reject the null hypothesis. The loop halts as soon as a p-value fails to fall below its target threshold.

Demšar [13] discusses Nemenyi's test for all pairwise comparisons and presents Holm's procedure mainly for comparisons against a control (pp. 11–15). Our implementation applies Holm's correction to all pairs. This is an adaptation. In our experiment we decided to select one configuration from each family, selecting the one with the highest mean accuracy. If two configurations have exactly the same accuracy, we choose the one with the lower mean prediction time. We applied this rule even when the statistical tests cannot identify a difference among the top configurations, so, selecting a configuration does not mean we have statistically demonstrated that it outperforms every alternative.

7.3 Stage one: top configuration selection

Table 4: Selected configurations and their performance

Selected hyperparameters

Dataset	Family	Parameters
wine	RF	criterion=entropy; trees=150; depth=20; max_features=sqrt; min_samples_leaf=1
wine	SVC	kernel=rbf; C=1; gamma=scale
wine	kNN	k=5; distance=ivdm; voting=majority; retention/forgetting=Never
pen-based	RF	criterion=entropy; trees=300; depth=40; max_features=sqrt; min_samples_leaf=1
pen-based	SVC	kernel=rbf; C=10; gamma=scale
pen-based	kNN	k=3; distance=euclidean; voting=inverse; retention/forgetting=OG

Performance of selected configurations

Dataset	Family	Accuracy (%)	Time (ms/case)
wine	RF	98.421 ± 2.542	0.358831 ± 0.020830
wine	SVC	99.474 ± 1.664	0.017147 ± 0.007945
wine	kNN	97.368 ± 4.473	0.138482 ± 0.011982
pen-based	RF	99.190 ± 0.295	0.046458 ± 0.000709
pen-based	SVC	99.636 ± 0.177	0.050430 ± 0.000526
pen-based	kNN	99.400 ± 0.211	0.516301 ± 0.233713

Table 4 shows the selected configurations for all three model families on both datasets. The complete selection records are provided in `selections/wine.csv` and `selections/pen-based.csv`. Accuracy and time are reported as mean $\pm$ sample standard deviation across the ten folds; these are not confidence intervals.

For SVC, both selected configurations use the RBF kernel and `gamma=scale`, but their regularization parameters differ: $C = 1$ for Wine and $C = 10$ for Pen-based.

For Random Forest, Wine uses 150 trees and a maximum depth of 20, whereas Pen-based uses 300 trees and a maximum depth of 40. Both selected configurations use `criterion=entropy`, `max_features=sqrt`, and `min_samples_leaf=1`. For each dataset, several Random Forest configurations shared the highest mean accuracy, so the one with the lowest mean prediction time was selected.

For kNN, Wine now uses $k = 5$, IVDM distance, majority voting, and the `Never` retention/forgetting strategy. Pen-based uses $k = 3$, Euclidean distance, inverse voting, and the `OG` retention/forgetting strategy. These choices follow the accuracy-first selection rule and do not, by themselves, establish statistically significant superiority over every alternative.

Table 5: Stage-one Friedman results

Dataset	Family	Accuracy p	Accuracy differences	Time p	Significant wins
wine	RF	0.222	No	0.0001	0/47
wine	SVC	0.0001	Yes	0.0001	12/34
wine	kNN	1	No	0.0001	0/95
pen-based	RF	0.0001	Yes	0.0001	6/47
pen-based	SVC	0.0001	Yes	0.0001	14/34
pen-based	kNN	0.0001	Yes	0.0001	24/95

Significant wins: competitors significantly outperformed on accuracy after Holm correction.

All time tests in this table are significant. The last column counts comparisons in which the selected configuration has significantly better ranks for accuracy.

For Wine, the Friedman test does not detect accuracy differences among RF configurations ($p = 0.222$) or among kNN configurations ($p = 1$). Therefore, we do not run accuracy post-hoc tests for these two families. Their configurations are chosen using our practical rule of selecting the highest accuracy, not because they have been proven to be better. SVC does show overall differences, but the selected configuration significantly outperforms only some of its competitors. For Pen-based, overall accuracy differences are detected in all three families.

7.4 Stage 2: Comparing models and instance reduction methods

After configuration selection, we keep the chosen RF, SVC, and kNN hyperparameters fixed. We evaluate each model family without reduction and with FCNN, ENNTh, and DROP4. We do not reconfigure the classifiers after reduction.

FCNN uses the selected distance and voting rule, with retention disabled, ENNTh and DROP4 use Euclidean distance and the selected k. The Wine run uses $k = 5$ for the reductions and IVDM within FCNN, whereas Pen-based uses $k = 3$ and Euclidean distance within FCNN. ENNTh uses $\mu = 0.8$ for both datasets.

Then, we repeat the statistical analysis. For each dataset, we run one Friedman test for accuracy with the 12 candidates and another for prediction time. When detecting a significant difference, we make all 66 pairwise comparisons using Holm's correction. We do not run additional overall tests just for the three unreduced models or for the four versions of each model family.

This gives us three ways to interpret the same comparison: differences between the original RF, SVC, and kNN models; the effect of each reduction method compared to the unreduced run of the same family; and differences between any other pair, including models from different families using different reduction

methods.

Below, we present two sections of a table per dataset. The first displays the mean percentage accuracy achieved across all 10 folds for a selected model combined with a reduction technique, as well as the average milliseconds taken to classify each case when running on a test set. The second shows, per classifier-reduction technique pair, the percentage of the training set remaining after reduction, the reduction time in seconds per fold, and the training time in seconds per fold.

Table 6: Final comparison on Wine Dataset

Predictive performance

Family	**Reduction**	**Accuracy (%)**	**Time (ms/case)**
SVC	ENNTh	99.474 ± 1.664	0.011538 ± 0.001335
SVC	No reduction	99.474 ± 1.664	0.011745 ± 0.000913
RF	No reduction	98.421 ± 2.542	0.365431 ± 0.035814
RF	ENNTh	97.895 ± 3.680	0.365500 ± 0.046081
kNN	No reduction	97.368 ± 4.473	0.130628 ± 0.003696
kNN	ENNTh	97.368 ± 4.473	0.135894 ± 0.021098
SVC	FCNN	96.692 ± 4.599	0.009595 ± 0.000674
kNN	FCNN	96.130 ± 6.526	0.103816 ± 0.003406
kNN	DROP4	95.110 ± 4.635	0.104696 ± 0.011885
RF	FCNN	95.019 ± 5.335	0.350115 ± 0.020749
RF	DROP4	93.744 ± 7.438	0.374528 ± 0.096017
SVC	DROP4	92.622 ± 6.163	0.010013 ± 0.002434

Storage and computational costs

Family	**Reduction**	**Retained set (%)**	**Reduction time (s/fold)**	**Training time (s/fold)**
SVC	ENNTh	91.70	0.0037	0.001319
SVC	No reduction	100.00	0.0000	0.001367
RF	No reduction	100.00	0.0000	0.137037
RF	ENNTh	91.70	0.0037	0.141101
kNN	No reduction	100.00	0.0000	0.000617
kNN	ENNTh	91.70	0.0037	0.000629
SVC	FCNN	18.28	0.1211	0.000945
kNN	FCNN	18.28	0.1211	0.000465
kNN	DROP4	16.05	0.0538	0.000496
RF	FCNN	18.28	0.1211	0.117337
RF	DROP4	16.05	0.0538	0.125468
SVC	DROP4	16.05	0.0538	0.000976

Table 7: Final comparison on Pen-based

Predictive performance

Family	Reduction	Accuracy (%)	Time (ms/case)
SVC	No reduction	99.636 ± 0.177	0.077301 ± 0.042644
SVC	ENNTh	99.491 ± 0.279	0.049054 ± 0.009252
kNN	No reduction	99.400 ± 0.211	0.522374 ± 0.186546
SVC	FCNN	99.390 ± 0.264	0.023419 ± 0.003204
kNN	ENNTh	99.236 ± 0.243	0.434110 ± 0.019417
RF	No reduction	99.190 ± 0.295	0.065467 ± 0.031565
SVC	DROP4	99.126 ± 0.328	0.018644 ± 0.000554
RF	ENNTh	98.936 ± 0.271	0.050329 ± 0.010990
kNN	DROP4	98.572 ± 0.418	0.050575 ± 0.000845
kNN	FCNN	98.135 ± 0.593	0.051440 ± 0.003170
RF	FCNN	98.008 ± 0.301	0.041535 ± 0.001376
RF	DROP4	96.434 ± 0.751	0.038973 ± 0.001448

Storage and computational costs

Family	Reduction	Retained set (%)	Reduction time (s/fold)	Training time (s/fold)
SVC	No reduction	100.00	0.0000	0.245655
SVC	ENNTh	98.38	4.6358	0.163026
kNN	No reduction	100.00	0.0000	0.000338
SVC	FCNN	5.27	6.0688	0.009469
kNN	ENNTh	98.38	4.6358	0.000267
RF	No reduction	100.00	0.0000	5.052120
SVC	DROP4	5.01	21.6329	0.008033
RF	ENNTh	98.38	4.6358	4.847885
kNN	DROP4	5.01	21.6329	0.000045
kNN	FCNN	5.27	6.0688	0.000036
RF	FCNN	5.27	6.0688	0.657661
RF	DROP4	5.01	21.6329	0.556314

Table 8: Friedman results for the final 12 candidates

Dataset	Metric	Friedman statistic	Permutation p	Candidates	Differences
wine	Accuracy	27.994	0.0018	12	Yes
wine	Time	105.769	0.0001	12	Yes
pen-based	Accuracy	91.503	0.0001	12	Yes
pen-based	Time	101.154	0.0001	12	Yes

The Friedman test for accuracy scores on the Wine dataset yields a permutation p-value of 0.0018. The other three p-values reach 0.0001, the minimum value supported by the simulation. All four tests detect differences within the corresponding group of 12 candidates. This does not mean every pair of candidates differs, or that the four effects are equally large.

On the Wine dataset, SVC with ENNTh and SVC without reduction share the highest final mean accuracy, 99.474%. The selection rule chooses SVC with EN-

NTh because its mean prediction time is lower: 0.011538 rather than 0.011745 ms/case. Neither the accuracy comparison nor the time comparison between these two candidates is significant after Holm correction. This is a tie-break, not evidence of differences. On the Pen-based dataset, SVC without reduction is the final practical choice, with mean accuracy of 99.636%. The post-hoc does not detect an accuracy difference between any pair of the three unreduced models in either dataset.

Table 9: Comparison of the unreduced models (pairs)

Dataset	Model pair	Accuracy p	Accuracy winner	Time p	Faster model
wine	RF vs. SVC	1	None detected	0.000136	SVC (unreduced)
wine	RF vs. kNN	1	None detected	0.472056	None detected
wine	SVC vs. kNN	1	None detected	0.76721	None detected
pen-based	RF vs. SVC	0.589071	None detected	1	None detected
pen-based	RF vs. kNN	1	None detected	0.417984	None detected
pen-based	SVC vs. kNN	1	None detected	1	None detected

P-values are Holm-adjusted. "None detected" means no statistically significant difference was detected.

"None detected" means that the test does not reject the null hypothesis of no difference for that pair but it does not prove equivalence.

On Wine, SVC is found to be faster than RF under this procedure. On Pen-based, none of the time comparisons between the three unreduced models is significant.

7.5 How reduction affects each original model

To make the results easier to read, our tables compare each reduced candidate with the unreduced model from the same family. Accuracy changes are expressed in percentage points: a negative value means accuracy decreased. Speedup is calculated as mean time of original / mean time of reduced model, so values greater than one indicate faster predictions.

Table 10: Changes relative to the original models on Wine

Family	Reduction	Accuracy (pp)	Speedup	Holm p (acc.)	Holm p (time)
RF	FCNN	-3.402	1.04×	1	1
RF	ENNTh	-0.526	1.00×	1	1
RF	DROP4	-4.677	0.98×	1	1
SVC	FCNN	-2.781	1.22×	1	1
SVC	ENNTh	0.000	1.02×	1	1
SVC	DROP4	-6.852	1.17×	0.039636	1
kNN	FCNN	-1.238	1.26×	1	1
kNN	ENNTh	0.000	0.96×	1	1
kNN	DROP4	-2.258	1.25×	1	1

Table 11: Changes relative to the original models on Pen-based

Family	Reduction	Accuracy (pp)	Speedup	Holm p (acc.)	Holm p (time)
RF	FCNN	-1.183	1.58×	0.077003	0.514043
RF	ENNTh	-0.255	1.30×	1	1
RF	DROP4	-2.757	1.68×	0.002163	0.149477
SVC	FCNN	-0.246	3.30×	1	0.000608
SVC	ENNTh	-0.146	1.58×	1	1
SVC	DROP4	-0.510	4.15×	0.231952	3.097×10^{-5}
kNN	FCNN	-1.265	10.16×	0.004129	0.417984
kNN	ENNTh	-0.164	1.20×	1	1
kNN	DROP4	-0.828	10.33×	0.032533	0.472056

In these comparisons, no reduction increases the mean accuracy of its original model. Significant accuracy differences favor the unreduced model. Significant prediction-time differences occur only on Pen-based, where they favor SVC with FCNN and SVC with DROP4.

On Wine, FCNN retains around 18.28% of the training data and DROP4 retains 16.05%, while ENNTh retains 91.70%. SVC with ENNTh and kNN with ENNTh match the mean accuracy of their respective unreduced models. In the original against reduced comparisons, the analysis detects an accuracy loss only for SVC with DROP4. For kNN, ENNTh is now the reduced version with the highest mean accuracy, while FCNN has the lowest mean prediction time. None of the variants are significantly different from the original kNN on accuracy or time after Holm correction.

On Pen-based, FCNN and DROP4 each retain about 5% of the training data, while ENNTh retains 98.38%. For kNN, FCNN and DROP4 reduce mean prediction time, with speedups of approximately 10.16× and 10.33×, respectively. Their mean accuracies decrease by 1.265 and 0.828 percentage points. The post-hoc analysis detects both losses. However, the average prediction-time differences for these pairs are not significant according the the post-hoc test. While this result is surprising at a glance given the magnitude of improvement, it is due to the comparison of average prediction-time rank across the 10 folds, rather than the absolute difference in average prediction times. This is a limitation of the Holm post-hoc approach rather than a conclusive assessment regarding the effect of the two reduction algorithms on the kNN configuration's average prediction time.

8 Discussion

The results show that the choice of model and reduction method depends on the dataset and on whether accuracy or prediction time matters most. We first selected one configuration per model and dataset. We then compared the three

selected models and their reduced versions, giving 12 candidates per dataset. Accuracy and time were analysed separately throughout.

SVC with an RBF kernel achieved the highest mean accuracy among the unreduced models: 99.47% on Wine and 99.64% on Pen-based. The selected value of C was 1 for Wine and 10 for Pen-based, with a gamma argument of "scale" proving optimal for both. However, the final Holm comparisons did not detect an accuracy difference between any pair of unreduced models on either dataset (Table 9).

The success of the RBF kernel over the linear kernel suggests the non-linear nature of its decision boundaries worked better than hyperplanes to separate classes in both datasets. The lower misclassification penalty used for the Wine dataset compared to the Pen-based dataset might indicate that decision boundaries were smoother for the former, and more complex for the latter. The "scale" gamma parameter causes the SVC to set gamma according to

$$\gamma = 1/(n_{\text{features}} \cdot \text{Var}(X))$$

where $\text{Var}(X)$ is the variance of all entries of the training set X taken together. When features are normalised to [0,1] ranges, $\text{Var}(X) \approx 1$ and so $\gamma \approx 1/(n_{\text{features}})$. Simplistically relating this back to the function of gamma, this might suggest training instances should have a smaller radius of influence as the dimensionality of the feature space increases. When examining the worst SVC results, the interplay between C and gamma values is clear: runs with low values of both C and gamma yielded mean accuracies of $< 40\%$ and $< 20\%$ on Wine and Pen-based folds respectively. With a smooth decision boundary due to small gamma and low misclassification penalties, these configurations likely suffered from underfitting.

For Random Forest, the selected configurations were different for each dataset. Wine used 150 trees with a maximum depth of 20, while Pen-based used 300 trees with a maximum depth of 40. One possible explanation is that Pen-based has more instances and classes, so having more trees and allowing greater depth might help the model learn more detailed patterns. Both datasets selected entropy, square-root feature sampling, and a minimum leaf size of 1. These settings influence how the trees split the data, allowing them to learn detailed patterns while introducing randomness through feature sampling, although we cannot confirm that this combination was better than the alternatives. Random Forest also required more training time than SVC, especially on Pen-based, which could be explained by the computational cost of building hundreds of decision trees. From an efficiency perspective, this suggests SVC may be a preferable choice of classifier.

For k-NN on the Wine dataset, the most accurate model used $k = 5$, IVDM, majority voting, and Never Retain, reaching 97.37% accuracy. The top performer on Pen-based used $k = 3$, Euclidean distance, inverse-distance voting, and Oblivion by Goodness, reaching 99.40%.These differences suggest that the

preferred hyperparameters depend on the characteristics of the classification problem. A larger k can reduce the influence of individual neighbors, while a smaller k focuses more on local patterns. Similarly, IVDM considers class information when measuring distance, while Euclidean distance measures numerical differences between instances. Inverse-distance voting gives more importance to closer neighbors, whereas majority voting treats all neighbors equally. The different retention strategies also suggest that updating the case base may be useful in some situations but not in others. However, these results do not prove that any individual hyperparameter is better. On Wine, no significant accuracy differences were found between the k-NN configurations. On Pen-based, significant differences were found overall, but the selected configuration significantly outperformed only 24 of the other 95 candidates. This suggests that several configurations may achieve comparable accuracy, making prediction time and storage requirements important factors when selecting a model.

Instance reduction offered the largest observed prediction speedups on the Pen-based dataset. FCNN and DROP4 retained only 5.27% and 5.01% of the training instances, respectively. For our selected k-NN configuration, mean prediction time per case fell from 0.522 ms to about 0.051 ms, giving speedups of 10.16× and 10.33×. Accuracy fell from 99.40% to 98.13% with FCNN and 98.57% with DROP4.

SVC gave a more favourable accuracy–speed trade-off on Pen-based. FCNN reached 99.39% accuracy and DROP4 reached 99.13%, compared with 99.64% without reduction. Their mean prediction speedups were 3.30× and 4.15×, and both time improvements were significant after Holm correction.

These savings also carry a preparation cost. On Pen-based, fitting SVC took about 0.246 s per fold without reduction and less than 0.010 s with FCNN or DROP4. However, reduction itself took about 6.07 s for FCNN and 21.63 s for DROP4. Faster prediction may justify this cost when the trained model is used many times or when training time is less important and fast predictions are prioritised. For a small number of predictions, the extra preparation can outweigh the savings.

On the Wine dataset, aggressive reduction offered smaller speedups and lower mean accuracy. For example, SVC with DROP4 fell to 92.62%, a significant loss relative to unreduced SVC. RF also lost mean accuracy with FCNN and DROP4 on both datasets. Its DROP4 loss on Pen-based was 2.76 percentage points and was significant after Holm correction. None of the RF prediction-time comparisons against its unreduced version was significant. These results do not support reduction as a general improvement for RF in this experiment.

ENNTh with removed far fewer instances than FCNN and DROP4 on both datasets, retaining 91.70% on Wine and 98.38% on Pen-based. On Wine, SVC with ENNTh matched the unreduced model's 99.47% accuracy and had a slightly lower mean prediction time. Our tie-breaking rule therefore selected

SVC with ENNTh. Neither its accuracy nor its time differed significantly from unreduced SVC. On Pen-based, unreduced SVC remained the final choice because it had the highest mean accuracy.

The varying extents of the reductions are notable. Despite the aggressive $\mu = 0.8$ value chosen for ENNTh, it offered a very modest reduction on both datasets. Given that it is an edition algorithm which attempts to remove noisy instances far from the core of each class, this would suggest that both datasets have relatively few peripheral or noisy instances. FCNN's condensation approach, conserving boundary instances and discarding interior ones, causes an aggressive reduction, as does DROP4's hybrid approach. This adds weight to the theory that most training instances for both datasets were located within the interior of their class regions.

9 Conclusions and limitations

This comparative study evaluated the trade-offs between classification accuracy, computational efficiency, and storage footprint across eager (SVC, RF) and lazy (k-NN) learning models, coupled with prototype reduction algorithms (FCNN, ENNTh, DROP4) on two benchmark datasets. Across all evaluations, unreduced SVC with an RBF kernel demonstrated the most consistent predictive performance, securing the highest average accuracy on both Wine (99.47%) and Pen-based (99.64%). Nonetheless, rigorous non-parametric analysis under Demšar's methodology indicated no statistically significant accuracy differences among the three unreduced model families, confirming that carefully tuned k-NN and RF baselines achieve competitive accuracy.

The primary distinction resides in computational latency. On large datasets, unreduced k-NN is constrained by query latency due to exhaustive pairwise distance computations. This is particularly pronounced when using the IVDM distance metric, which is more computationally intensive than the others. Condensation and hybrid reduction (FCNN and DROP4) resolve this bottleneck by pruning roughly 95% of training instances, yielding over 8× inference speedups for k-NN at the expense of a minor, statistically detectable accuracy penalty (approximately 0.8–1.3 percentage points).

Our final choices are therefore SVC with ENNTh for Wine and unreduced SVC for Pen-based under the stated selection rule. Reduction remains useful when a smaller training set or faster prediction is worth some loss in mean accuracy and extra preparation time. There is no single reduction method that improves every model on both datasets.

A key limitation on the strength of these conclusions is the fact that, for both datasets, there were several models that achieved very high average accuracies, to the extent that overall top performers were not highly distinguished from their

peers. Additionally, several methodological constraints should be considered when interpreting these findings:

1. **Fold-Level Block Dependence:** Cross-validation folds served as blocks within Demšar's Friedman test. Because training sets overlap across folds, the assumption of independent blocks is violated, which can underestimate variance and inflate Type I error rates relative to tests over distinct datasets.
2. **Conservative Multiple Testing Penalty:** Applying Holm's step-down correction across all 66 candidate pairs within a 10-fold sample structure imposes a strict family-wise error threshold. While protecting against false discoveries, this conservative adjustment increases Type II errors, limiting the statistical power to distinguish closely performing configurations.
3. **Asymptotic Post-Hoc Approximation:** While the Friedman omnibus test employed 9,999 permutations, post-hoc rank comparisons relied on normal-distribution z-approximations, which are asymptotic and exhibit reduced reliability with only ten observations per candidate.
4. **Parameter Selection and Post-Reduction Retuning:** The ENNTh filtering threshold ($\mu = 0.8$) was chosen after exploratory evaluations on baseline data rather than through nested cross-validation. Furthermore, model hyperparameters were held fixed after reduction rather than re-optimized over the reduced datasets.
5. **Preprocessing and Scaling Bounds:** Min-max normalization parameters were derived exclusively from training sets without clipping test instances to $[0, 1]$. While preserving outlier magnitude without distortion, test observations occasionally fell outside the unit hypercube, slightly altering Euclidean boundary proportions.

References

[1] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, *Wine Quality*, UCI Machine Learning Repository, 2009. DOI: 10.24432/C56S3T

[2] E. Alpaydin and F. Alimoglu, *Pen-Based Recognition of Handwritten Digits*, UCI Machine Learning Repository, DOI: https://doi.org/10.24432/C5MG6K, 1996.

[3] D. R. Wilson and T. R. Martinez, "Improved heterogeneous distance functions," *Journal of Artificial Intelligence Research*, vol. 6, pp. 1–34, 1997. DOI: 10.1613/jair.346

[4] F. Angiulli, "Fast nearest neighbor condensation for large data sets classification," *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 11, pp. 1450–1464, 2007. DOI: 10.1109/TKDE.2007.190645

[5] F. Vázquez, J. S. Sánchez, and F. Pla, "A stochastic approach to Wilson's editing algorithm," in *Pattern Recognition and Image Analysis*, ser. Lecture Notes in Computer Science, vol. 3523, Springer, 2005, pp. 35–42. DOI: 10.1007/11492542_5

[6] D. R. Wilson and T. R. Martinez, "Reduction techniques for instance-based learning algorithms," *Machine Learning*, vol. 38, no. 3, pp. 257–286, Mar. 2000. DOI: 10.1023/A:1007626913721

[7] scikit-learn developers. "Support vector machines," Accessed: Oct. 24, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/svm.html

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324

[9] D. W. Aha, D. Kibler, and M. K. Albert, "Instance-based learning algorithms," *Machine Learning*, vol. 6, no. 1, pp. 37–66, Jan. 1991. DOI: 10.1023/a:1022689900470

[10] P. Hart, "The condensed nearest neighbor rule (corresp.)," *IEEE Transactions on Information Theory*, vol. 14, no. 3, pp. 515–516, May 1968. DOI: 10.1109/tit.1968.1054155

[11] D. L. Wilson, "Asymptotic properties of nearest neighbor rules using edited data," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-2, no. 3, pp. 408–421, Jul. 1972. DOI: 10.1109/TSMC.1972.4309137

[12] A. Jung, *A gentle introduction to supervised machine learning*, arXiv preprint arXiv:1805.05052v1, 2018. [Online]. Available: https://arxiv.org/abs/1805.05052v1

[13] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. [Online]. Available: https://jmlr.org/papers/v7/demsar06a.html

[14] *Introduction to machine learning. work 2: Classification with lazy learning, SVM and RF*, Course assignment. Author, institution and year not provided.

[15] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, Morgan Kaufmann, 1995, pp. 1137–1145. [Online]. Available: https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf